Auditoría Técnica del Backend Generado con IA

Objetivo
El estudiante deberá evaluar de forma crítica el código que la IA generó, como lo haría un arquitecto o revisor técnico, no como un simple programador.

Instrucciones
Con base en el código generado usando Cursor IA:

Checklist de Revisión del Backend Generado con IA

1. Revisión de la estructura del módulo

Ítem	Criterio de revisión	✓ Cumple	⚠ Parcial	❌ No cumple	Observaciones
1.1	Existe la estructura models.py / schemas.py / services.py / routes.py				
1.2	No hay mezcla de responsabilidades entre archivos				
1.3	routes.py no contiene lógica de negocio				
1.4	El acceso a base de datos ocurre solo en services.py				
1.5	El módulo representa correctamente su dominio				

2. Evaluación de principios de diseño

2.1 Single Responsibility Principle (SRP)

Ítem	Criterio de revisión	✓	⚠	❌	Observaciones
2.1.1	Cada archivo tiene una única responsabilidad				
2.1.2	Cada función cumple un solo propósito				
2.1.3	No existen funciones con múltiples responsabilidades				
2.1.4	Cambios en reglas afectan un solo punto del código				

2.2 DRY (Don’t Repeat Yourself)

Ítem	Criterio de revisión	✓	⚠	✗	Observaciones
2.2.1	No existe lógica duplicada				
2.2.2	Validaciones comunes están centralizadas				
2.2.3	No se repiten consultas innecesarias				
2.2.4	Funciones y clases tienen nombres coherentes y únicos				

2.3 Clean Architecture

Ítem	Criterio de revisión	✓	⚠	✗	Observaciones
2.3.1	Las rutas solo orquestan la petición				
2.3.2	Los servicios contienen la lógica del dominio				
2.3.3	Los modelos ORM no contienen lógica de negocio				
2.3.4	Las dependencias fluyen hacia el dominio				
2.3.5	El módulo podría cambiar de framework sin romperse				

2.4 Tipado explícito

Ítem	Criterio de revisión	✓	⚠	✗	Observaciones
2.4.1	Todas las funciones tienen tipado de entrada y salida				
2.4.2	No se usa Any sin justificación				
2.4.3	Esquemas Pydantic usan tipos correctos				
2.4.4	Relaciones ORM están correctamente tipadas				

3. Identificación de resultados

3.1 Aciertos detectados

Ítem	Descripción	Observaciones
3.1.1	Acierto 1 identificado	
3.1.2	Acierto 2 identificado	

3.2 Errores detectados

Ítem	Descripción	Observaciones
3.2.1	Error 1 identificado	
3.2.2	Error 2 identificado	

3.3 Riesgos futuros

Ítem	Riesgo identificado	Impacto	Observaciones
3.3.1	Riesgo técnico	Alto / Medio / Bajo	

4. Conclusión técnica

Resultado final	Marcar
Apto para producción	
Requiere refactorización menor	
Requiere refactorización estructural	
No cumple estándares profesionales	

Actividad: Trazabilidad Modelo ER → Código

Objetivo
Cerrar el ciclo **diseño → implementación**, algo que normalmente no se enseña bien.

Actividad
Construir una tabla:

Entidad ER	Modelo ORM	Schema Pydantic	Endpoint
USUARIO	User	UserCreate	POST /users

Y responder:

- ¿Qué campo del ER NO aparece aún?
- ¿Qué relación falta modelar?
- ¿Qué endpoint debería existir y no existe?

Actividad: Análisis de Seguridad del Backend Generado

Objetivo
Elevar el nivel hacia **ingeniería segura**, no solo funcional.

Preguntas guía

- ¿Se exponen datos sensibles?
- ¿Las entradas están validadas?
- ¿Hay control de acceso?
- ¿Qué ataque OWASP podría afectar este endpoint?

Formato:

- Lista de vulnerabilidades
- Propuesta de mitigación (conceptual)

Actividad: Refactorización Conceptual (sin tocar código)

Objetivo
Desarrollar **pensamiento arquitectónico**, no mecanográfico.

Actividad
Desarrollar:

- Dibuje (a mano o digital) el flujo:
- Request → Schema → Service → ORM → DB
- Indique dónde puede romperse el sistema
- Marque responsabilidades críticas

Reflexión guiada sobre uso de IA

Pregunta clave
¿Qué parte del desarrollo **NO debería delegarse jamás a una IA** en un sistema profesional?
Evaluar:

- Criterio técnico
- Responsabilidad profesional
- Ética en ingeniería